

Comparación de Métodos de Codificación

September 15, 2025

1 Genetic Algorithm (GA)

- Autor: Rodrigo S. Cortez-Madrigal
- web: <https://roicort.github.io>
- github: <https://github.com/roicort/MetaZoo>
- docs: <https://roicort.github.io/MetaZoo/>

1.1 Objetivo

Evaluar empíricamente el efecto de la codificación del individuo (binaria, real y entera) sobre el desempeño de un algoritmo evolutivo al resolver problemas paramétricos y combinatorios. Se requiere un análisis de resultados, así como conclusiones.

1.1.1 2) Problemas a resolver

2.1. Paramétricos

- Esfera, dimensión $N=5$
- Eggholder

2.2. Combinatorios (enteros)

- TSP (Traveling Salesman Problem) con 17 ciudades. (Se proporciona la matriz de distancias)
- Criterio: minimizar la longitud total del tour hamiltoniano (circuito).
- N-Reinas para $N=8$ y $N=12$.
- Criterio: minimizar el número de conflictos (ataques) entre reinas.

1.1.2 3) Codificaciones requeridas

- Binaria (para todos los problemas)
- Real (para todos los problemas)
- Entera (solo TSP y N-Reinas)

Mantenga fijos para todos los experimentos los métodos de selección y elitismo.

1.2 Codificaciones

```
[1]: import numpy as np
import pandas as pd
import plotly.graph_objects as go
from plotly.subplots import make_subplots
```

```

from metazoo.bio.evolutionary import GeneticAlgorithm
from metazoo.bio.evolutionary.operators import selection, mutation, crossover

import plotly.io as pio
#pio.renderers.default = "jupyterlab"
pio.renderers.default = "svg"

```

Las codificaciones ya implementadas son la binaria y real para problemas paramétricos, y la permutación para problemas combinatorios.

```

[2]: from metazoo.bio.evolutionary.utils import encoding

#binaryencoder = encoding.Binary()
#realencoder = encoding.Real()
#permutationencoder = encoding.Permutation()

```

Tendremos que implementar las siguientes codificaciones:

- Binaria para permutaciones (TSP y N-Reinas)
- Real para (TSP y N-Reinas)

Esto porque no creo que valga la pena implementar la codificación entera para los problemas combinatorios, ya que no es la más adecuada.

Para esto usamos la clase abstracta `Encoding` de `MetaZoo`.

```

[3]: from metazoo.bio.evolutionary.utils import Encoding

class BinaryPermutation(Encoding):
    """
    Codificación binaria para permutaciones.
    In a binary representation of the permutation, each element is encoded as a
    ↪string
    of  $\lceil \log_2 n \rceil$  bits, an individual is a string of  $n \lceil \log_2 n \rceil$  bits.
    """

    def __init__(self, permutation_size: int):
        super().__init__()
        self.bits_per_element = int(np.ceil(np.log2(permutation_size)))
        self.length = permutation_size

    def encode(self, population_size: int) -> np.ndarray:
        total_bits = self.length * self.bits_per_element
        population = np.random.randint(0, 2, (population_size, total_bits))
        return population

```

```

def decode(self, individual: np.ndarray) -> np.ndarray:
    decoded = []
    for i in range(self.length):
        start = i * self.bits_per_element
        end = start + self.bits_per_element
        element_bits = individual[start:end]
        element_value = int("".join(map(str, element_bits)), 2)
        decoded.append(element_value % self.length) # Hack para que esté
    en rango
    # Eliminar duplicados y rellenar con los faltantes
    unique_elements = list(dict.fromkeys(decoded))
    missing_elements = [i for i in range(self.length) if i not in
    unique_elements]
    # Rellenar con los faltantes
    permutation = unique_elements + missing_elements
    return np.array(permutation)[:self.length]

class RealPermutation(Encoding):

    def __init__(self, permutation_size: int):
        super().__init__()
        self.length = permutation_size

    def encode(self, population_size: int) -> np.ndarray:
        # Cada individuo es un vector de números reales entre 0 y 1
        return np.random.rand(population_size, self.length)

    def decode(self, individual: np.ndarray) -> np.ndarray:
        # La permutación se obtiene ordenando los valores reales
        return individual.argsort()

```

1.3 Problemas Paramétricos

Aquí compararemos el desempeño de las codificaciones binaria y real en los problemas paramétricos.

```
[4]: np.random.seed(42)
```

```

[5]: from metazoo.gym.mono import Function

gens = 100

for fitness in [Function('EggHolder'), Function('Sphere')]:
    print(f"Function: {fitness.__name__}")

    if fitness == Function('Sphere'):
        bounds = [(-5.12, 5.12), (-5.12, 5.12), (-5.12, 5.12), (-5.12, 5.12),
    (-5.12, 5.12)]

```

```

else:
    bounds = fitness.bounds

binaryencoder = encoding.Binary(precision=3, bounds=bounds)
realencoder = encoding.Real(bounds=bounds)

gaBinary = GeneticAlgorithm(
    fitness_function=fitness,
    crossover_function=crossover.onepoint,
    mutation_function=mutation.flip_bit,
    selection_function=selection.rank,
    population_size=100,
    mutation_rate=0.01,
    crossover_rate=0.7,
    encoder=binaryencoder,
    elitism=0.1,
    minimize=True,
)

gaBinary.run(generations=gens, history=False, verbose=False)
#fig = gaBinary.fitness_plot()
#fig.update_layout(title_text=f"Fitness - Binary Encoding [{fitness.
↪ __name__}]")
#fig.show()

gaReal = GeneticAlgorithm(
    fitness_function=fitness,
    crossover_function=crossover.onepoint,
    mutation_function=mutation.gaussian,
    selection_function=selection.rank,
    population_size=100,
    mutation_rate=0.01,
    crossover_rate=0.7,
    encoder=realencoder,
    elitism=0.1,
    minimize=True,
)

gaReal.run(generations=gens, history=False, verbose=False)
#fig = gaReal.fitness_plot()
#fig.update_layout(title_text=f"Fitness - Real Encoding [{fitness.
↪ __name__}]")
#fig.show()

# Comparación de históricos

# Graficar el mejor fitness por generación para ambas codificaciones

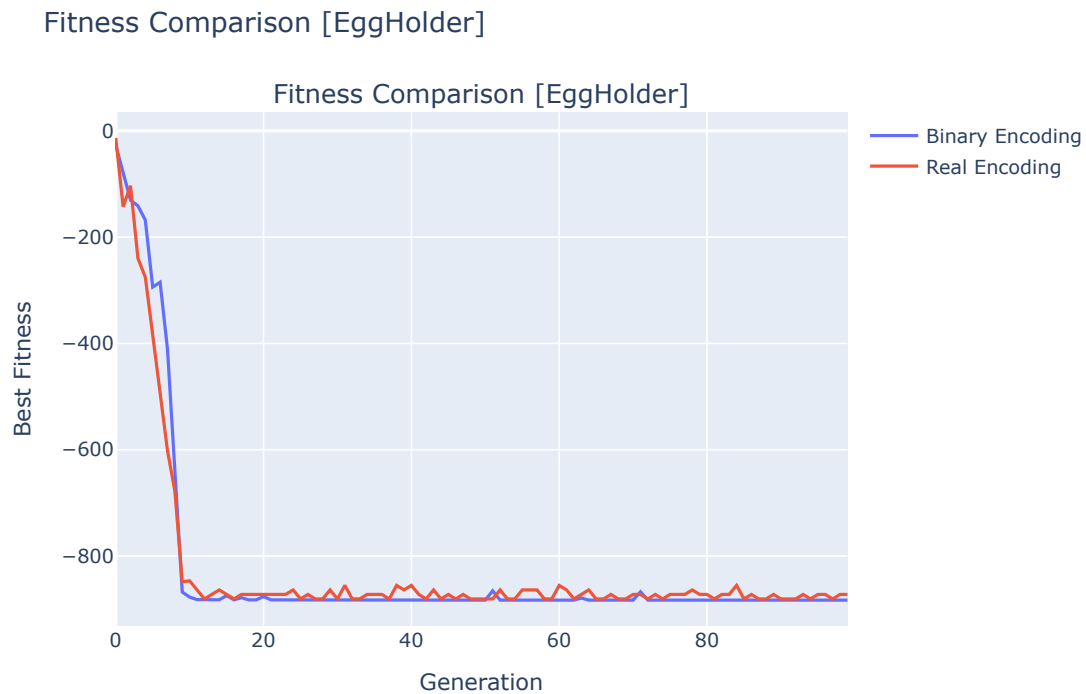
```

```

fig = make_subplots(rows=1, cols=1, subplot_titles=[f"Fitness Comparison_{fitness.__name__}"])
fig.add_trace(go.Scatter(
    y=np.array(gaBinary.fitness_history),
    mode='lines',
    name='Binary Encoding'
), row=1, col=1)
fig.add_trace(go.Scatter(
    y=np.array(gaReal.fitness_history),
    mode='lines',
    name='Real Encoding'
), row=1, col=1)
fig.update_xaxes(title_text="Generation", row=1, col=1)
fig.update_yaxes(title_text="Best Fitness", row=1, col=1)
fig.update_layout(title_text=f"Fitness Comparison [{fitness.__name__}]")
fig.show()

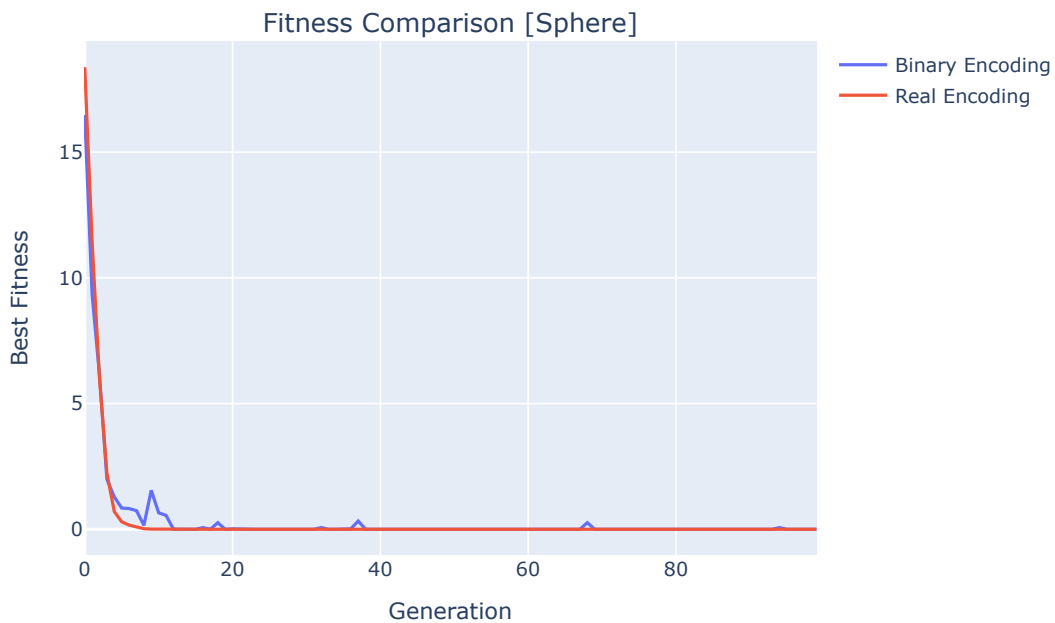
```

Function: EggHolder



Function: Sphere

Fitness Comparison [Sphere]



Podemos notar que la codificación real es mucho más eficiente que la binaria, ya que converge más rápido en general. La codificación real permite una exploración más directa del espacio de soluciones, mientras que la codificación binaria puede llegar a requerir más generaciones para llegar a la solución óptima, además de que la precisión está limitada por el número de bits usados en la representación. Y además, mientras más precisión necesitemos, también se incrementará el número de bits y por lo tanto también el tiempo de cómputo.

1.4 Problemas Combinatorios

1.4.1 TSP

```
[6]: from metazoo.gym.combinatorial import TSP
```

```
[7]: # Para la tarea se requiere utilizar un TSP del que solo tenemos una matriz de
      ↪distancias
```

```
# Matriz de distancias del TSP (17 ciudades)
```

```
distance_matrix = np.array([
    [0,633,257,91,412,150,80,134,259,505,353,324,70,211,268,246,121],
    [633,0,390,661,227,488,572,530,555,289,282,638,567,466,420,745,518],
    [257,390,0,228,169,112,196,154,372,262,110,437,191,74,53,472,142],
    [91,661,228,0,383,120,77,105,175,476,324,240,27,182,239,237,84],
```

```

[412,227,169,383,0,267,351,309,338,196,61,421,346,243,199,528,297],
[150,488,112,120,267,0,63,34,264,360,208,329,83,105,123,364,35],
[80,572,196,77,351,63,0,29,232,444,292,297,47,150,207,332,29],
[134,530,154,105,309,34,29,0,249,402,250,314,68,108,165,349,36],
[259,555,372,175,338,264,232,249,0,495,352,95,189,326,383,202,236],
[505,289,262,476,196,360,444,402,495,0,154,578,439,336,240,685,390],
[353,282,110,324,61,208,292,250,352,154,0,435,287,184,140,542,238],
[324,638,437,240,421,329,297,314,95,578,435,0,254,391,448,157,301],
[70,567,191,27,346,83,47,68,189,439,287,254,0,145,202,289,55],
[211,466,74,182,243,105,150,108,326,336,184,391,145,0,57,426,96],
[268,420,53,239,199,123,207,165,383,240,140,448,202,57,0,483,153],
[246,745,472,237,528,364,332,349,202,685,542,157,289,426,483,0,336],
[121,518,142,84,297,35,29,36,236,390,238,301,55,96,153,336,0]
])

# Definimos un TSP personalizado usando la matriz de distancias

custom_tsp = TSP.Custom(distance_matrix=distance_matrix)

encoderInt = encoding.Permutation(permutation_size=custom_tsp.dimension)
encoderReal = RealPermutation(permutation_size=custom_tsp.dimension)
encoderBin = BinaryPermutation(permutation_size=custom_tsp.dimension)

history = {}

for encoder, crossover_method, mutation_method in [
    (encoderInt, crossover.PMX, mutation.swap),
    (encoderReal, crossover.onepoint, mutation.gaussian),
    (encoderBin, crossover.onepoint, mutation.flip_bit)
]:
    print(f"Encoder: {type(encoder).__name__}")

    gatsp = GeneticAlgorithm(
        fitness_function=custom_tsp,
        crossover_function=crossover_method,
        mutation_function=mutation_method,
        selection_function=selection.tournament,
        population_size=500,
        mutation_rate=0.05,
        crossover_rate=0.7,
        encoder=encoder,
        elitism=0.1,
        minimize=True,
    )

    gatsp.run(generations=100, verbose=False)
    history[type(encoder).__name__] = gatsp.fitness_history

```

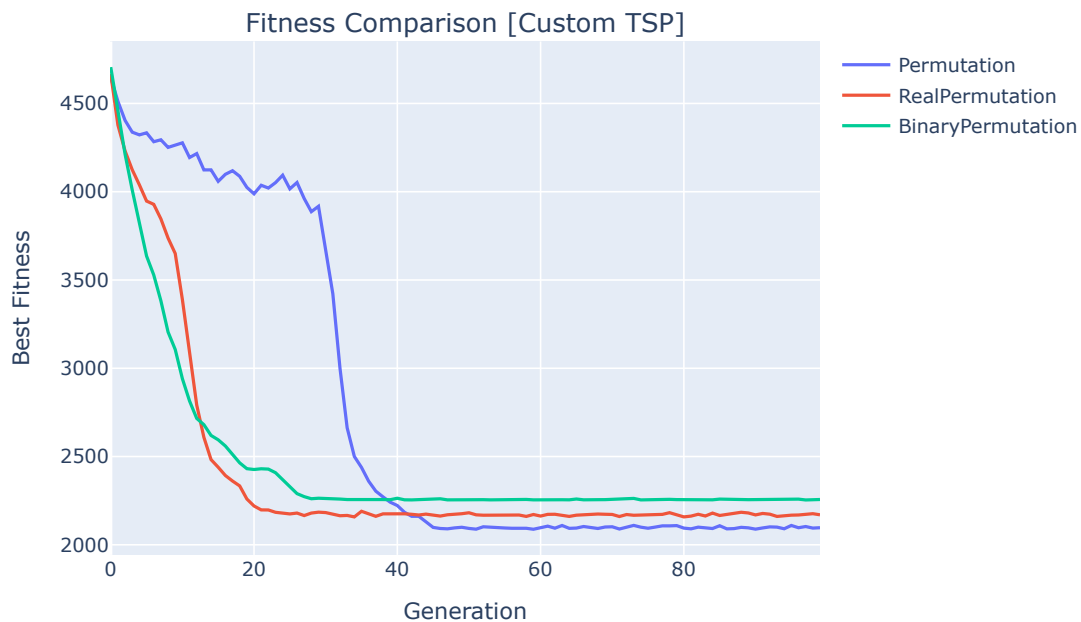
```

historydf = pd.DataFrame(history)

fig = make_subplots(rows=1, cols=1, subplot_titles=[f"Fitness Comparison_
↳[Custom TSP]"])
for col in historydf.columns:
    fig.add_trace(go.Scatter(
        y=historydf[col],
        mode='lines',
        name=col
    ), row=1, col=1)
fig.update_xaxes(title_text="Generation", row=1, col=1)
fig.update_yaxes(title_text="Best Fitness", row=1, col=1)
fig.show()

```

Encoder: Permutation
Encoder: RealPermutation
Encoder: BinaryPermutation



Aquí podemos notar que a pesar de que la codificación entera tarda un poco más en converger, es la que encuentra soluciones de mejor calidad (menor distancia total del tour) y es más consistente.

Otros tipos de codificación, como la binaria y la real, no alcanzan soluciones tan óptimas como la codificación entera. Supongo que es porque hay mas ruido en este tipo de codificaciones. Además para el tipo de codificación binaria para permutaciones, hay problemas con la representación que podría contener valores binarios que no representan permutaciones válidas, porque si el número de ciudades no es una potencia de 2, entonces hay combinaciones binarias que se quedan sin asignar o que representan números fuera del rango de índices de las ciudades.

Probemos ahora con un TSP con 52 ciudades, Berlin52. Con este problema más grande que es parte del benchmark TSPLIB y del cual se tiene una mejor solución conocida (distancia total = 7542), espero que se note aún más la diferencia entre las codificaciones.

```
[8]: tsp = TSP.Berlin52()

encoderInt = encoding.Permutation(permutation_size=tsp.dimension)
encoderReal = RealPermutation(permutation_size=tsp.dimension)
encoderBin = BinaryPermutation(permutation_size=tsp.dimension)

history = {}
best_solutions = {}

for encoder, crossover_method, mutation_method in [
    (encoderInt, crossover.PMX, mutation.swap),
    (encoderReal, crossover.onepoint, mutation.gaussian),
    (encoderBin, crossover.onepoint, mutation.flip_bit)
]:

    gatsp = GeneticAlgorithm(
        fitness_function=tsp,
        crossover_function=crossover_method,
        mutation_function=mutation_method,
        selection_function=selection.tournament,
        population_size=500,
        mutation_rate=0.05,
        crossover_rate=0.7,
        encoder=encoder,
        elitism=0.1,
        minimize=True,
    )

    gatsp.run(generations=1000, verbose=False)
    print("*" * 40)
    print(type(encoder).__name__)
    print(f'Best Individual: {gatsp.best_individual}')
    print(f'Best Fitness: {gatsp.best_fitness}')
    print(f'Optimal Value: {tsp.optimal_value}')
    print(f'Difference from Optimal: {gatsp.best_fitness - tsp.optimal_value}')
    history[type(encoder).__name__] = gatsp.fitness_history
    best_solutions[type(encoder).__name__] = gatsp.best_individual
```

```

historydf = pd.DataFrame(history)

fig = make_subplots(rows=1, cols=1, subplot_titles=[f"Fitness Comparison_
↳[Berlin52 TSP]"])
for col in historydf.columns:
    fig.add_trace(go.Scatter(
        y=historydf[col],
        mode='lines',
        name=col
    ), row=1, col=1)
fig.update_xaxes(title_text="Generation", row=1, col=1)
fig.update_yaxes(title_text="Best Fitness", row=1, col=1)
fig.show()

```

Permutation

Best Individual: [41 20 30 22 19 49 29 28 13 51 10 15 45 47 23 37 4 14 5 3 24
26 12 46
25 27 11 50 32 42 9 8 7 40 18 44 2 16 17 31 48 43 33 34 35 38 39 36
0 21 1 6]

Best Fitness: 9991.024308846592

Optimal Value: 7542.0

Difference from Optimal: 2449.0243088465922

RealPermutation

Best Individual: [23 43 21 48 31 0 17 34 47 15 1 6 20 8 46 13 10 50 39 18 11
22 19 51
12 26 14 3 44 40 7 35 45 24 32 42 41 29 28 33 5 27 25 49 16 2 30 37
38 9 36 4]

Best Fitness: 19190.842257697517

Optimal Value: 7542.0

Difference from Optimal: 11648.842257697517

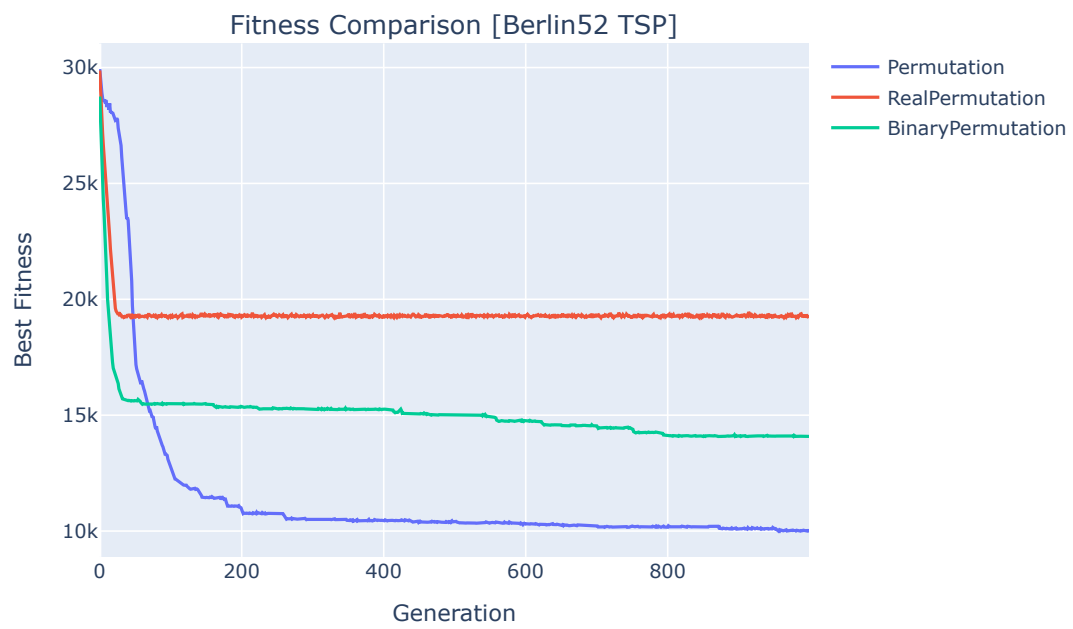
BinaryPermutation

Best Individual: [9 8 7 6 41 2 19 15 23 37 44 18 17 21 0 29 22 49 11 50 10
12 51 13
46 20 1 16 40 32 42 4 5 14 3 24 25 26 27 28 30 31 33 34 35 36 38 39
43 45 47 48]

Best Fitness: 14079.37503991661

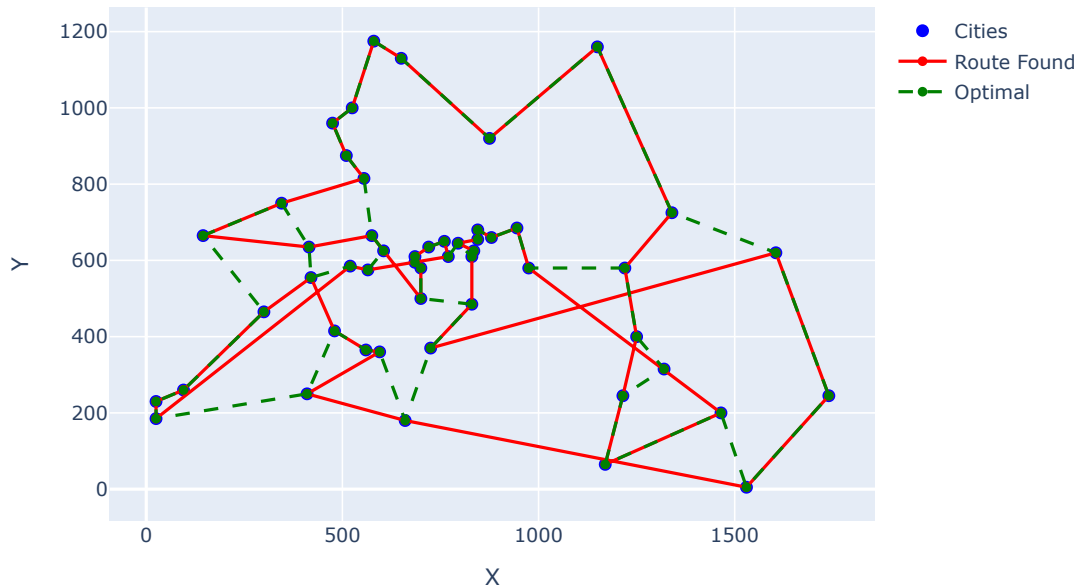
Optimal Value: 7542.0

Difference from Optimal: 6537.375039916609



```
[9]: tsp.plot(show_optimal=True, solution=
        best_solutions['Permutation'])
```

TSP: Berlin52



Efectivamente, la codificación entera sigue siendo la que mejor desempeño tiene, encontrando soluciones cercanas a la óptima conocida, en este caso con una diferencia de 2008 unidades de distancia.

1.4.2 N-Reinas

Para este problema, la función de *fitness* se define como el número de pares de reinas que **no** se atacan entre sí. Por lo tanto, el objetivo del algoritmo es *maximizar* este valor, recordemos que para esta implementación el algoritmo genético siempre maximiza la función de *fitness*.

$$\text{fitness} = \frac{N(N-1)}{2} - \text{número de ataques}$$

Donde N es el número de reinas y “número de ataques” es el total de pares de reinas que se atacan mutuamente en la solución propuesta.

Por esto usamos `minimize=False` en la configuración del GA.

```
[13]: from metazoo.gym.combinatorial import NQueens

for N in [8, 12, 24, 48]:
    print(f"N-Queens: {N}x{N}")
    nqueens = NQueens(n=N)
```

```

history = {}
best_solutions = {}

encoderInt = encoding.Permutation(permutation_size=N)
encoderReal = RealPermutation(permutation_size=N)
encoderBin = BinaryPermutation(permutation_size=N)

for encoder, crossover_method, mutation_method in [
    (encoderInt, crossover.PMX, mutation.swap),
    (encoderReal, crossover.onepoint, mutation.gaussian),
    (encoderBin, crossover.onepoint, mutation.flip_bit)
]:

    ganq = GeneticAlgorithm(
        fitness_function=nqueens,
        crossover_function=crossover_method,
        mutation_function=mutation_method,
        selection_function=selection.tournament,
        population_size=200,
        mutation_rate=0.01,
        crossover_rate=0.7,
        encoder=encoder,
        elitism=0.1,
        minimize=False,
    )

    ganq.run(generations=500, verbose=False)

    print("*" * 40)
    print(type(encoder).__name__)
    print(f'Best Individual: {ganq.best_individual}')
    print(f'Best Fitness: {ganq.best_fitness}')
    history[type(encoder).__name__] = ganq.fitness_history
    best_solutions[type(encoder).__name__] = ganq.best_individual

    nfig = nqueens.plot(solution=ganq.best_individual, attacks=nqueens.
↪attacks(ganq.best_individual))
    nfig.show()

    historydf = pd.DataFrame(history)

    fig = make_subplots(rows=1, cols=1, subplot_titles=[f"Fitness Comparison_
↪[N-Queens {N}x{N}]]")
    for col in historydf.columns:
        fig.add_trace(go.Scatter(
            y=historydf[col],
            mode='lines',

```

```

        name=col
    ), row=1, col=1)
fig.update_xaxes(title_text="Generation", row=1, col=1)
fig.update_yaxes(title_text="Best Fitness", row=1, col=1)
fig.show()

```

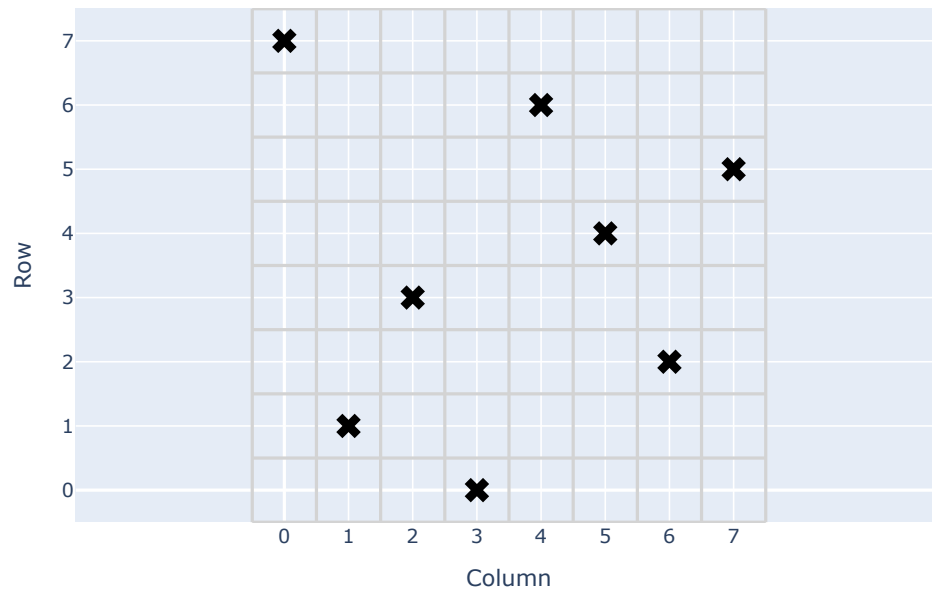
N-Queens: 8x8

Permutation

Best Individual: [7 1 3 0 6 4 2 5]

Best Fitness: 28.0

8-Queens

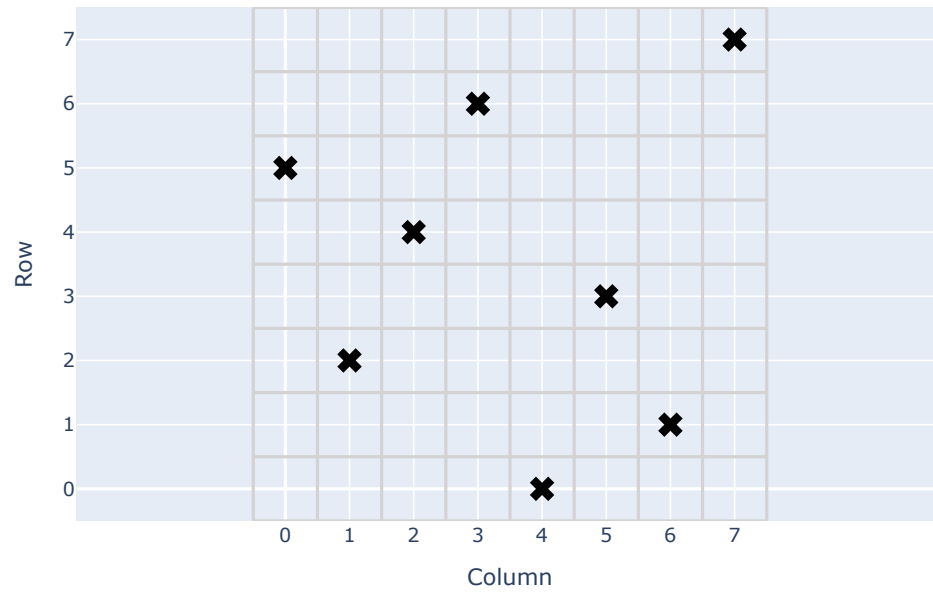


RealPermutation

Best Individual: [5 2 4 6 0 3 1 7]

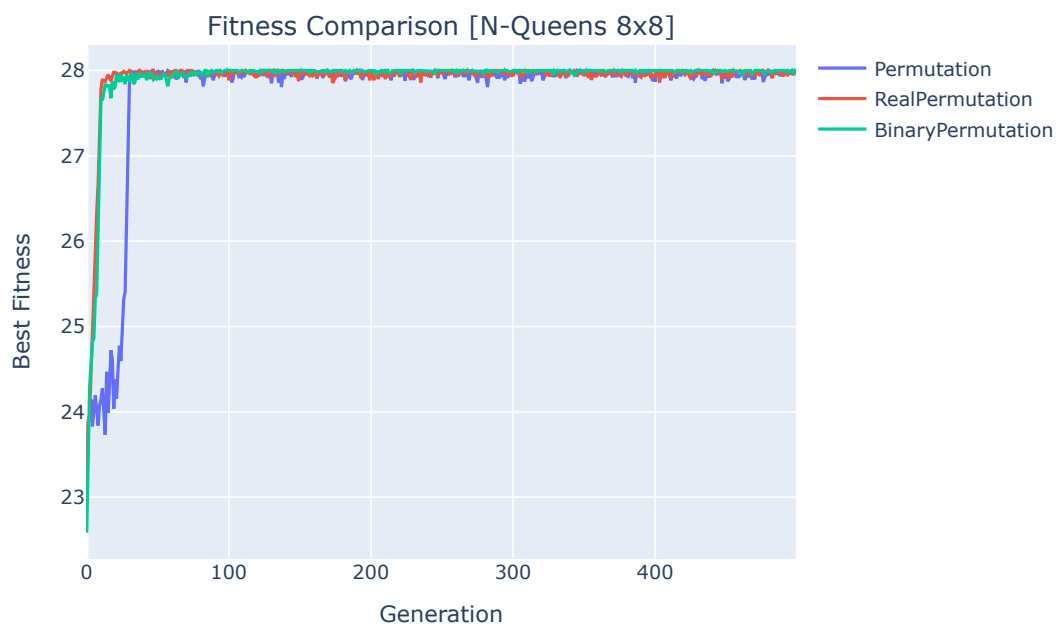
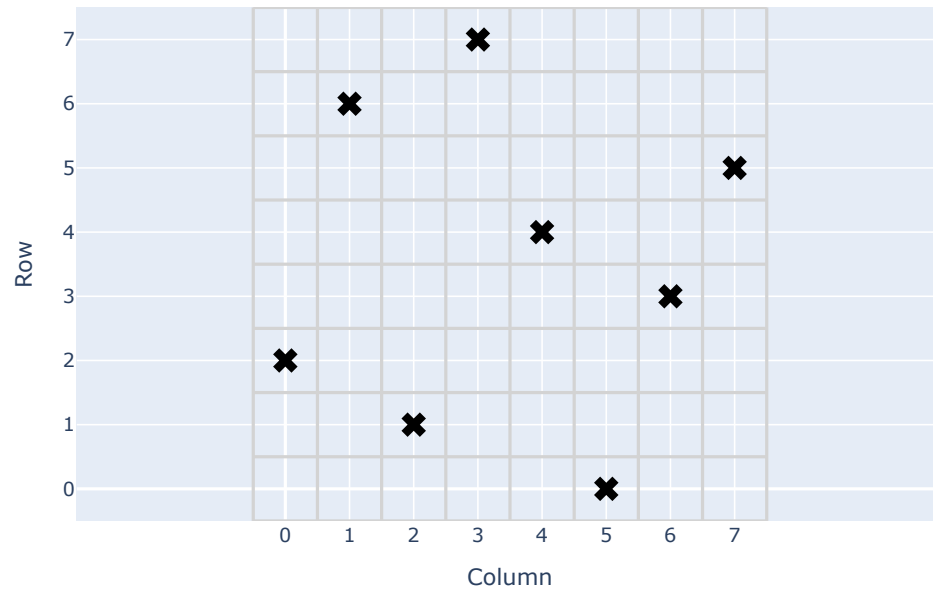
Best Fitness: 28.0

8-Queens



```
*****  
BinaryPermutation  
Best Individual: [2 6 1 7 4 0 3 5]  
Best Fitness: 28.0
```

8-Queens



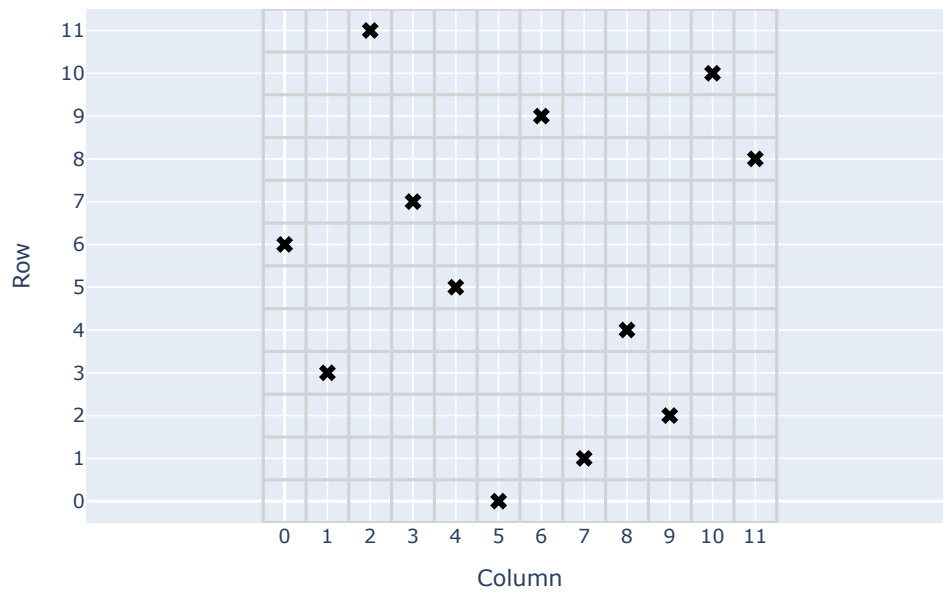
N-Queens: 12x12

Permutation

Best Individual: [6 3 11 7 5 0 9 1 4 2 10 8]

Best Fitness: 66.0

12-Queens

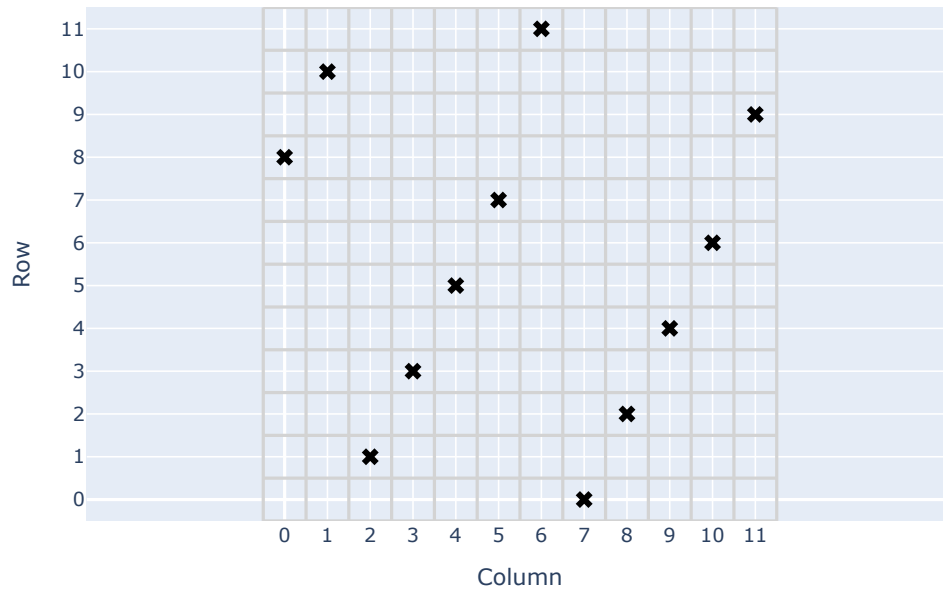


RealPermutation

Best Individual: [8 10 1 3 5 7 11 0 2 4 6 9]

Best Fitness: 66.0

12-Queens

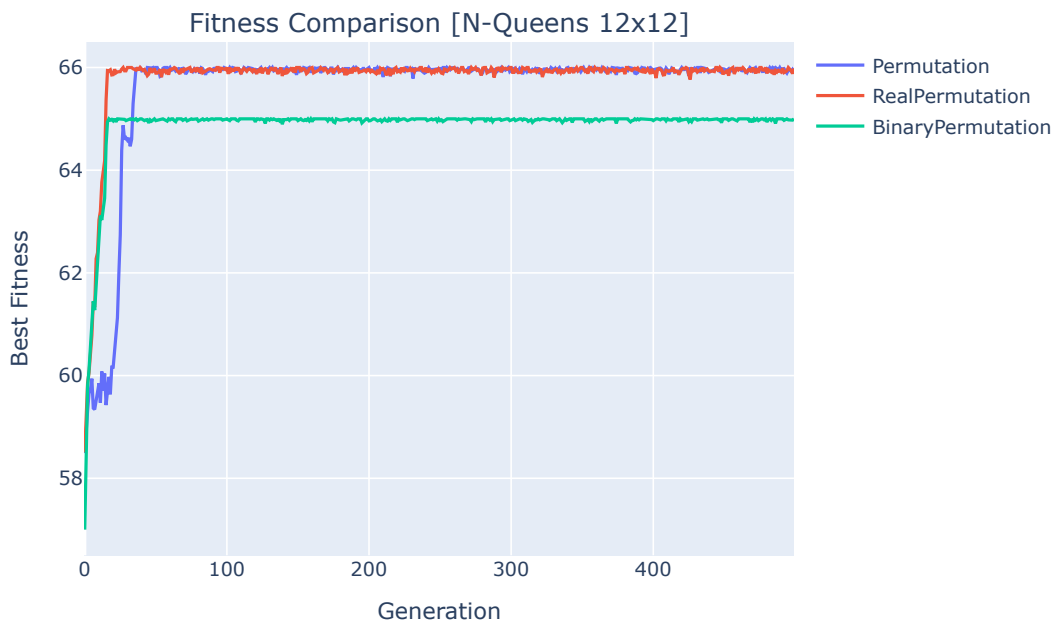
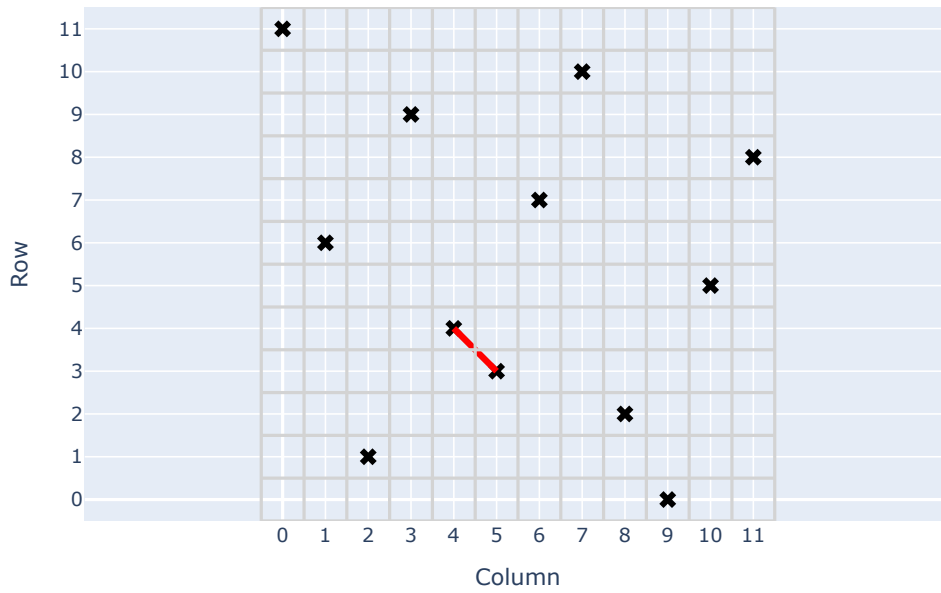


BinaryPermutation

Best Individual: [11 6 1 9 4 3 7 10 2 0 5 8]

Best Fitness: 65.0

12-Queens



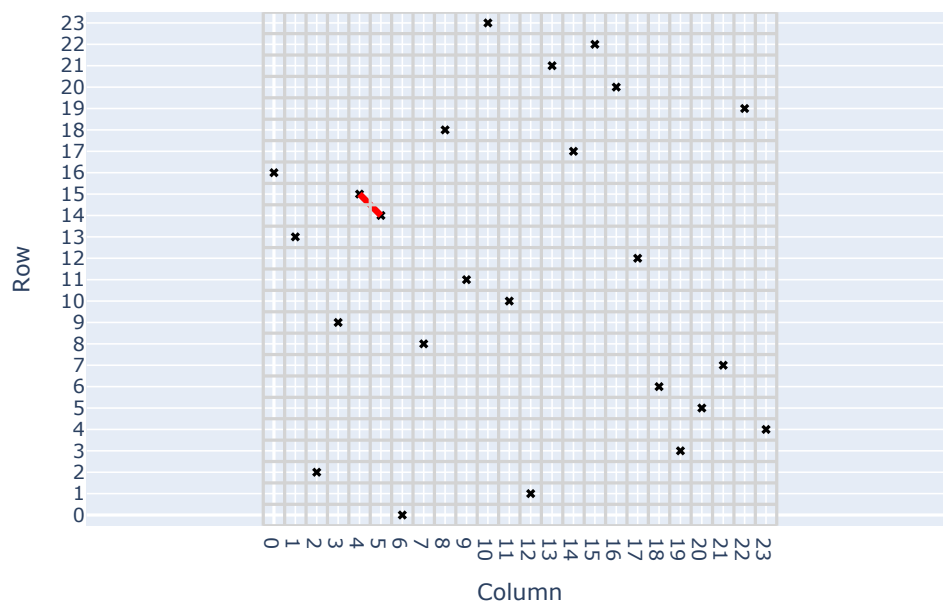
N-Queens: 24x24

Permutation

Best Individual: [16 13 2 9 15 14 0 8 18 11 23 10 1 21 17 22 20 12 6 3 5
7 19 4]

Best Fitness: 275.0

24-Queens

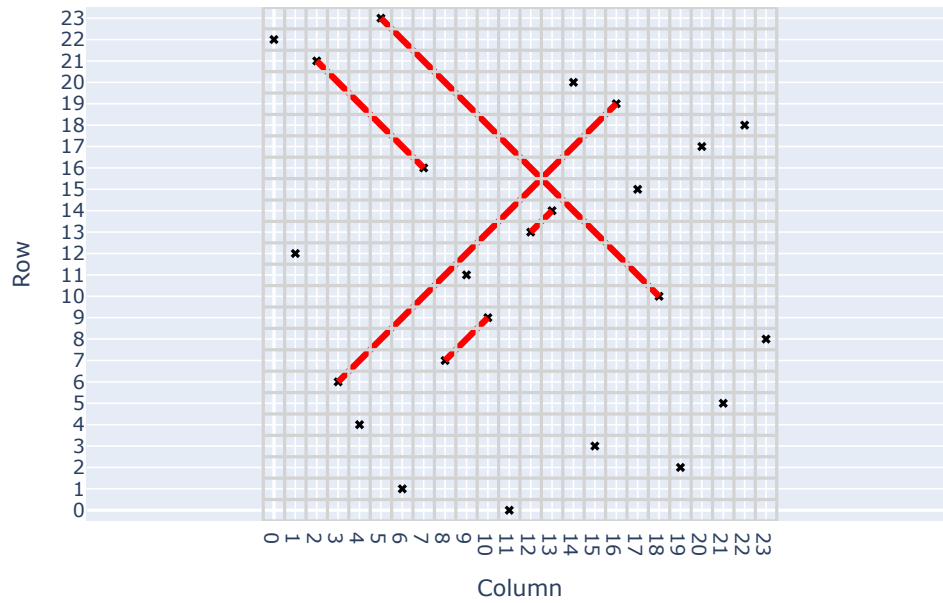


RealPermutation

Best Individual: [22 12 21 6 4 23 1 16 7 11 9 0 13 14 20 3 19 15 10 2 17
5 18 8]

Best Fitness: 271.0

24-Queens

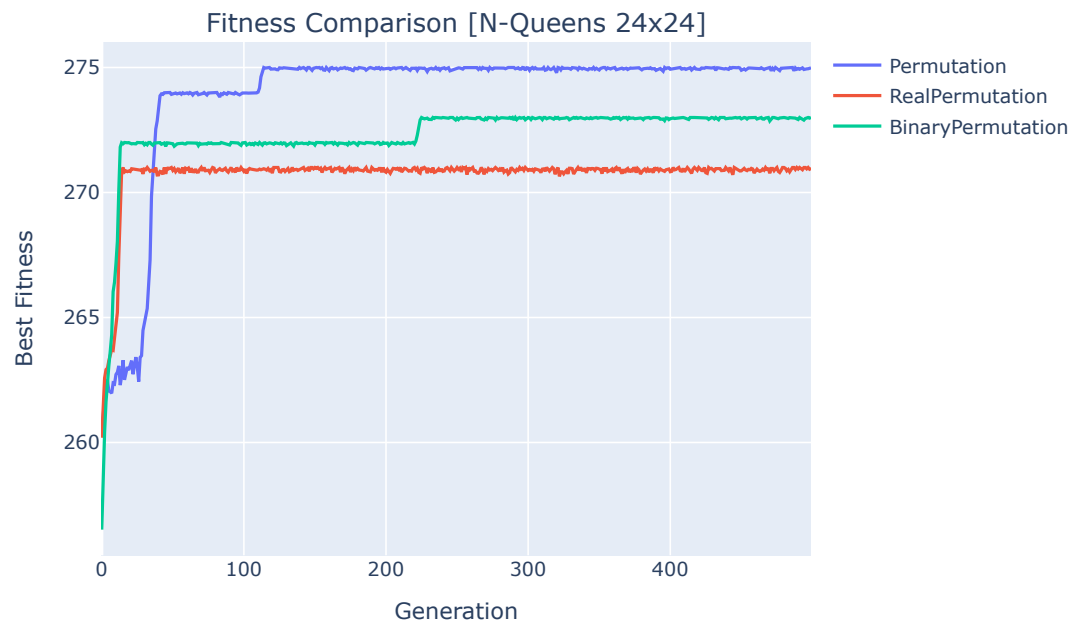
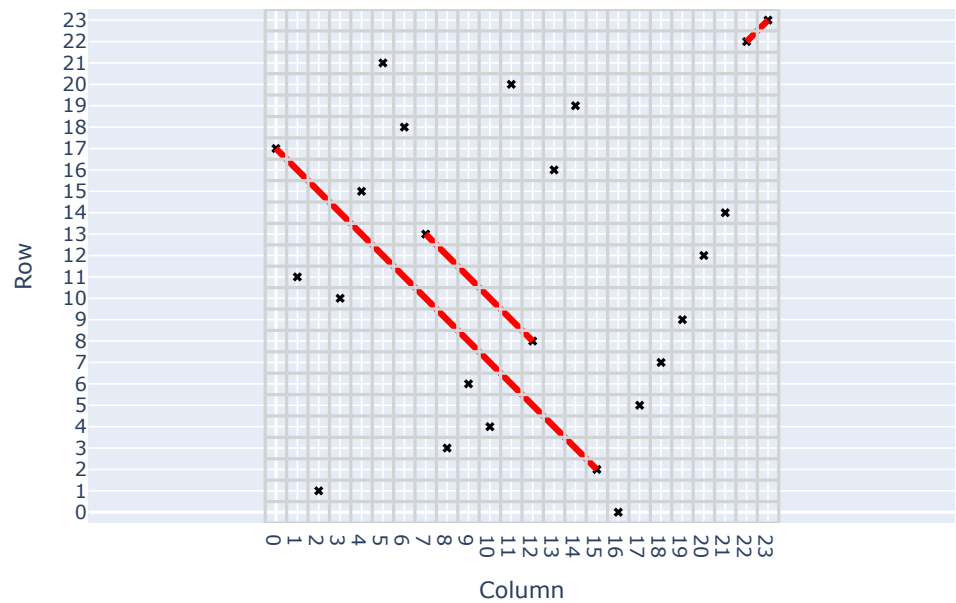


BinaryPermutation

Best Individual: [17 11 1 10 15 21 18 13 3 6 4 20 8 16 19 2 0 5 7 9 12
14 22 23]

Best Fitness: 273.0

24-Queens



N-Queens: 48x48

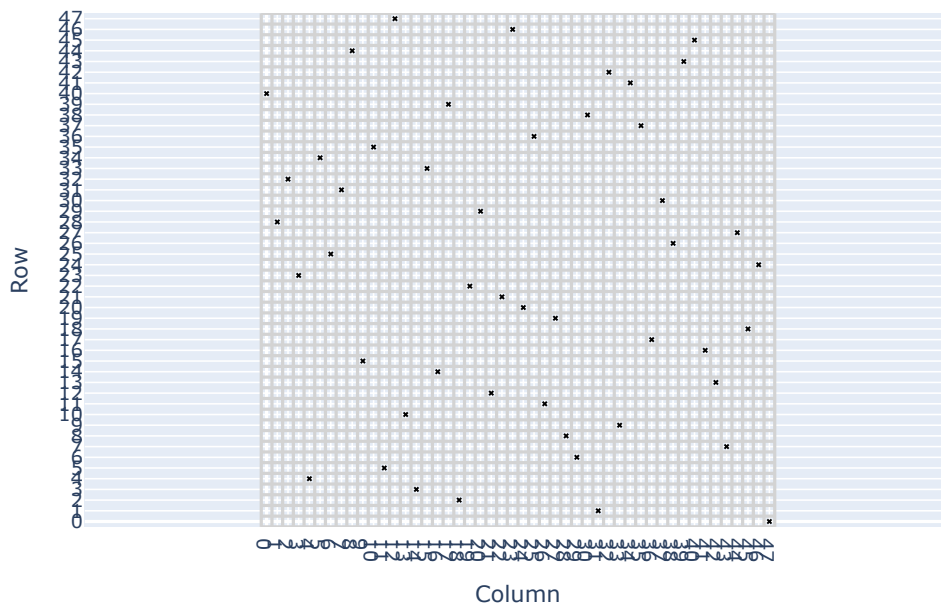
Permutation

Best Individual: [40 28 32 23 4 34 25 31 44 15 35 5 47 10 3 33 14 39 2 22 29
12 21 46

20 36 11 19 8 6 38 1 42 9 41 37 17 30 26 43 45 16 13 7 27 18 24 0]

Best Fitness: 1128.0

48-Queens



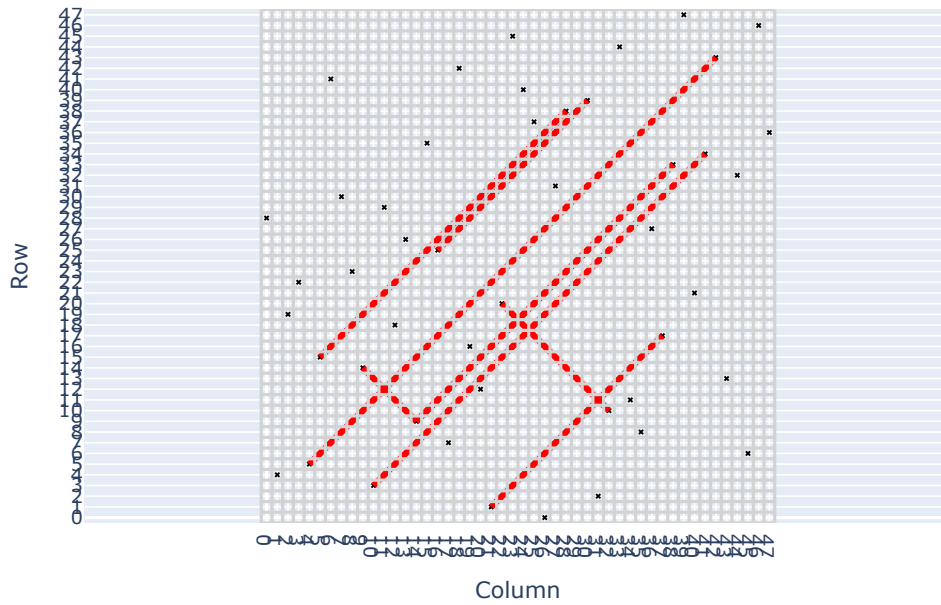
RealPermutation

Best Individual: [28 4 19 22 5 15 41 30 23 14 3 29 18 26 9 35 25 7 42 16 12
1 20 45

40 37 0 31 38 24 39 2 10 44 11 8 27 17 33 47 21 34 43 13 32 6 46 36]

Best Fitness: 1118.0

48-Queens



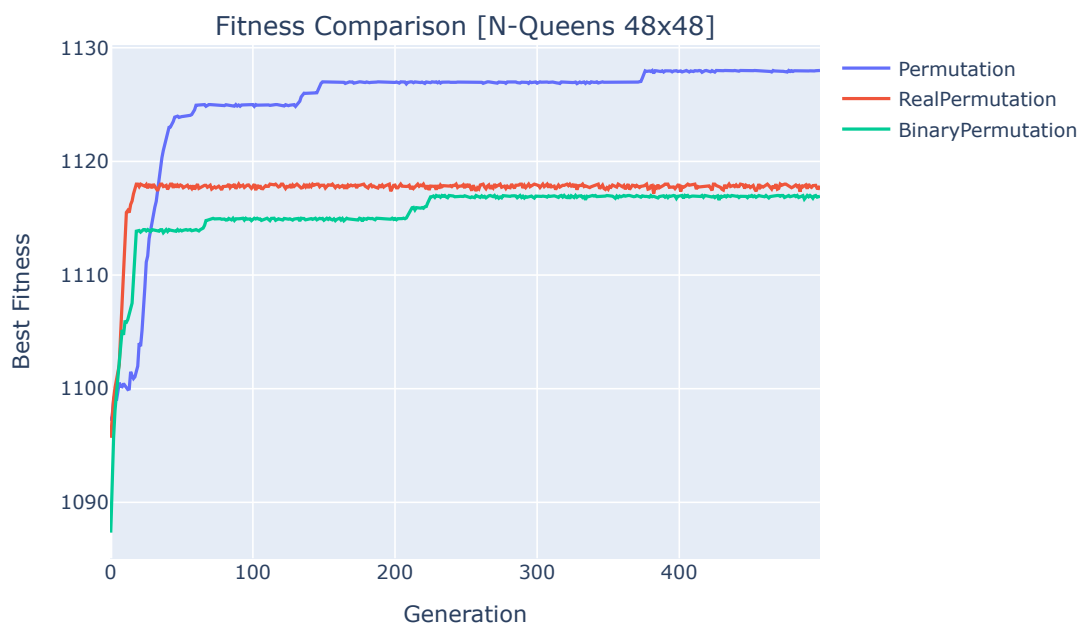
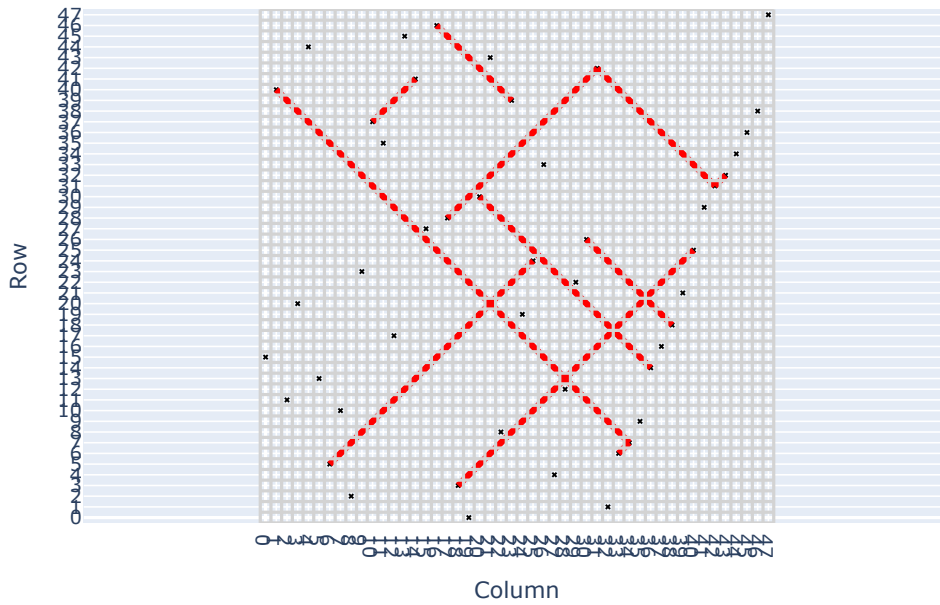
BinaryPermutation

Best Individual: [15 40 11 20 44 13 5 10 2 23 37 35 17 45 41 27 46 28 3 0 30
43 8 39

19 24 33 4 12 22 26 42 1 6 7 9 14 16 18 21 25 29 31 32 34 36 38 47]

Best Fitness: 1117.0

48-Queens



Aquí también es notable que para este problema la codificación Permutation (entera) es la que mejor funciona, sobre todo cuando tenemos un mayor número de reinas ($N=24$, $N=48$). Las otras codificaciones no logran encontrar soluciones óptimas en estos casos.